

Colby Crutcher, Ben Foster, Riley Rudolfo

Demo Video:

- In the video the executable is called mergesort, but has since changed it to ‘project’ per the rubric.
- The bash file is also different, as we now test different block sizes

<https://youtu.be/3UuTGXjHtrQ>

CUDA Merge Sort

This project implements a parallel merge sort on the GPU using a binary search-based approach[cite: 141, 253]. It compares the execution time of the GPU implementation against a standard sequential CPU merge sort to demonstrate parallel speedup.

Results

Array Size (N)	Block Size	CPU Time (ms)	GPU Time (ms)	Speedup
1,024	128	0.0415 ms	0.0317 ms	1.31x
65,536	128	3.3914 ms	0.1132 ms	29.96x
1,048,576	128	54.4669 ms	1.6507 ms	33.00x
1,024	256	0.0432 ms	0.0463 ms	0.93x
65,536	256	3.3841 ms	0.1708 ms	19.81x
1,048,576	256	55.0269 ms	2.4901 ms	22.10x
1,024	512	0.0473 ms	0.1075 ms	0.44x
65,536	512	3.3668 ms	0.2898 ms	11.62x
1,048,576	512	54.9337 ms	4.2245 ms	13.00x

Figure 1: Table

- The GPU is slower for small arrays because the overhead of launching parallel kernels and managing device memory outweighs the actual computation time, whereas the CPU can process the data instantly within its high-speed local cache. Other than that we are getting massive speedups for larger data sets.

How to Build

To compile the project, simply run the following command in your terminal:

make

Once compiled, you can run the program with its default settings (Array Size: 1024, Block Size: 256) by typing:

```
./project
```

Testing Custom Sizes: You can also pass custom array sizes and block sizes directly in the terminal to see how the GPU handles larger datasets:

```
./project [ArraySize] [BlockSize]
```

Example (testing ~1 million elements):

```
./project 1048576 256
```

- Note - the total array size (N) must be evenly divisible by the number of blocks (numBlocks).

I did this so that we don't have padding, and to avoid out of bounds queries.

arg2 (Block Size) is The number of threads per CUDA block. This defines the initial "tile size" for the shared memory odd-even sort and the subsequent merge steps.

After that, you will see the results printed to sort_comparison.txt

ex:

```
Array Size (N): 1024 | Block Size: 256
CPU Sequential Merge Sort Time: 0.062373 ms
GPU Parallel Merge Sort Time: 0.016384 ms
Speedup: 3.80695x
```

I also made a bash file that tests different array lengths. To use this, you may have to run:

```
chmod +x run_tests.sh
```

then,

```
./run_tests.sh
```

The results are also printed to sort_comparisons.txt

Cleanup

To clean the file, run 'make clean' - This deletes the executable project, and the sort_comparison.txt

Screenshots

```
ccrutter@csee-gpu01:~/FinalProj$ make
nvcc -arch=sm_75 -std=c++17 -O2 main.cu kernel.cu cpumergesort.cpp -o project
ccrutter@csee-gpu01:~/FinalProj$
```

Figure 2: Makefile

```
ccrutter@csee-gpu01:~/FinalProj$ ./project 1048576 256
Running FULL Merge Sort with N = 1048576, Block Size = 256
-----
Saved unsorted array to unsorted.txt
Saved CPU sorted array to cpu_sorted.txt
Saved GPU sorted array to gpu_sorted.txt
Appended timing results to sort_comparison.txt
ccrutter@csee-gpu01:~/FinalProj$
```

Figure 3: Sample Run

```
FinalProj > ≡ sort_comparison.txt
1 Array Size (N): 1048576 | Block Size: 256
2 CPU Sequential Merge Sort Time: 54.486 ms
3 GPU Parallel Merge Sort Time: 2.67965 ms
4 Speedup: 20.3333x
5 -----
6
```

Figure 4: Output

```

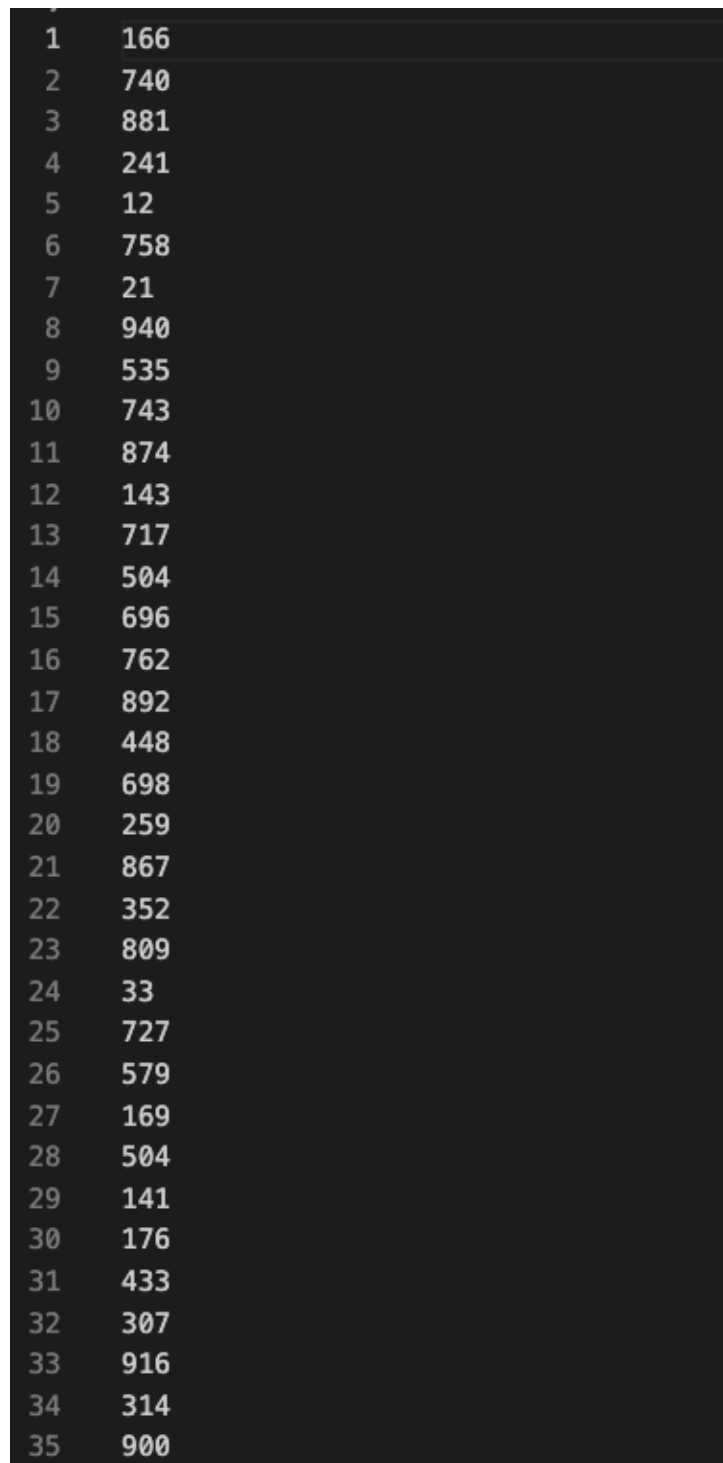
ccrutter@csee-gpu01:~/FinalProj$ ./run_tests.sh
Testing N=1024 with BlockSize=128
Running FULL Merge Sort with N = 1024, Block Size = 128
-----
Saved unsorted array to unsorted.txt
Saved CPU sorted array to cpu_sorted.txt
Saved GPU sorted array to gpu_sorted.txt
Appended timing results to sort_comparison.txt
Testing N=65536 with BlockSize=128
Running FULL Merge Sort with N = 65536, Block Size = 128
-----
Saved unsorted array to unsorted.txt
Saved CPU sorted array to cpu_sorted.txt
Saved GPU sorted array to gpu_sorted.txt
Appended timing results to sort_comparison.txt
Testing N=1048576 with BlockSize=128
Running FULL Merge Sort with N = 1048576, Block Size = 128
-----
Saved unsorted array to unsorted.txt
Saved CPU sorted array to cpu_sorted.txt
Saved GPU sorted array to gpu_sorted.txt
Appended timing results to sort_comparison.txt
Testing N=1024 with BlockSize=256
Running FULL Merge Sort with N = 1024, Block Size = 256
-----
Saved unsorted array to unsorted.txt
Saved CPU sorted array to cpu_sorted.txt
Saved GPU sorted array to gpu_sorted.txt
Appended timing results to sort_comparison.txt
Testing N=65536 with BlockSize=256
Running FULL Merge Sort with N = 65536, Block Size = 256
-----
Saved unsorted array to unsorted.txt
Saved CPU sorted array to cpu_sorted.txt
Saved GPU sorted array to gpu_sorted.txt
Appended timing results to sort_comparison.txt
Testing N=1048576 with BlockSize=256
Running FULL Merge Sort with N = 1048576, Block Size = 256
-----
Saved unsorted array to unsorted.txt
Saved CPU sorted array to cpu_sorted.txt
Saved GPU sorted array to gpu_sorted.txt
Appended timing results to sort_comparison.txt
Testing N=1024 with BlockSize=512
Running FULL Merge Sort with N = 1024, Block Size = 512
-----
Saved unsorted array to unsorted.txt
Saved CPU sorted array to cpu_sorted.txt
Saved GPU sorted array to gpu_sorted.txt
Appended timing results to sort_comparison.txt
Testing N=65536 with BlockSize=512
Running FULL Merge Sort with N = 65536, Block Size = 512

```

Figure 5: Bash Test File Run

```
Array Size (N): 1048576 | Block Size: 256
CPU Sequential Merge Sort Time: 54.486 ms
GPU Parallel Merge Sort Time: 2.67965 ms
Speedup: 20.3333x
-----
Array Size (N): 1024 | Block Size: 128
CPU Sequential Merge Sort Time: 0.042334 ms
GPU Parallel Merge Sort Time: 0.031584 ms
Speedup: 1.34036x
-----
Array Size (N): 65536 | Block Size: 128
CPU Sequential Merge Sort Time: 3.33813 ms
GPU Parallel Merge Sort Time: 0.114208 ms
Speedup: 29.2285x
-----
Array Size (N): 1048576 | Block Size: 128
CPU Sequential Merge Sort Time: 56.0159 ms
GPU Parallel Merge Sort Time: 1.65693 ms
Speedup: 33.8071x
-----
Array Size (N): 1024 | Block Size: 256
CPU Sequential Merge Sort Time: 0.043259 ms
GPU Parallel Merge Sort Time: 0.048128 ms
Speedup: 0.898832x
-----
Array Size (N): 65536 | Block Size: 256
CPU Sequential Merge Sort Time: 3.36797 ms
GPU Parallel Merge Sort Time: 0.172032 ms
Speedup: 19.5776x
-----
Array Size (N): 1048576 | Block Size: 256
CPU Sequential Merge Sort Time: 55.6716 ms
GPU Parallel Merge Sort Time: 2.49078 ms
Speedup: 22.3511x
-----
```

Figure 6: Bash Results



1	166
2	740
3	881
4	241
5	12
6	758
7	21
8	940
9	535
10	743
11	874
12	143
13	717
14	504
15	696
16	762
17	892
18	448
19	698
20	259
21	867
22	352
23	809
24	33
25	727
26	579
27	169
28	504
29	141
30	176
31	433
32	307
33	916
34	314
35	900

Figure 7: Unsorted.txt

```
FinalProj > ≡ cpu_sorted.txt
1 0
2 2
3 2
4 2
5 3
6 3
7 3
8 4
9 5
10 5
11 6
12 7
13 8
14 8
15 12
16 12
17 13
18 13
19 16
20 16
21 18
22 18
23 19
24 19
25 21
26 21
27 21
28 21
29 22
30 22
31 23
32 24
33 26
34 26
35 27
36 28
```

Figure 8: cpu_sorted.txt with 1024, and a 256 blocksize

FinalProj	>	≡	gpu_sorted.txt
1			0
2			2
3			2
4			2
5			3
6			3
7			3
8			4
9			5
10			5
11			6
12			7
13			8
14			8
15			12
16			12
17			13
18			13
19			16
20			16
21			18
22			18
23			19
24			19
25			21
26			21
27			21
28			21
29			22
30			22
31			23
32			24
33			26
34			26
35			27
36			28

Figure 9: gpu_sorted.txt with same input size and blocksize